Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	9 March 2026
Team ID	SWUID202641986
Project Name	Project - Automobile Insurance Fraud Detection System
Maximum Marks	4 Marks

Technical Architecture: AI-Powered Fraud Detection Backend
This document outlines the technical design of the Insurance Fraud Detection system, following the industry-standard IBM Developer pattern format.
Architectural Diagram

The following diagram illustrates the interaction between the client, the application server, and the AI/ML backend components.

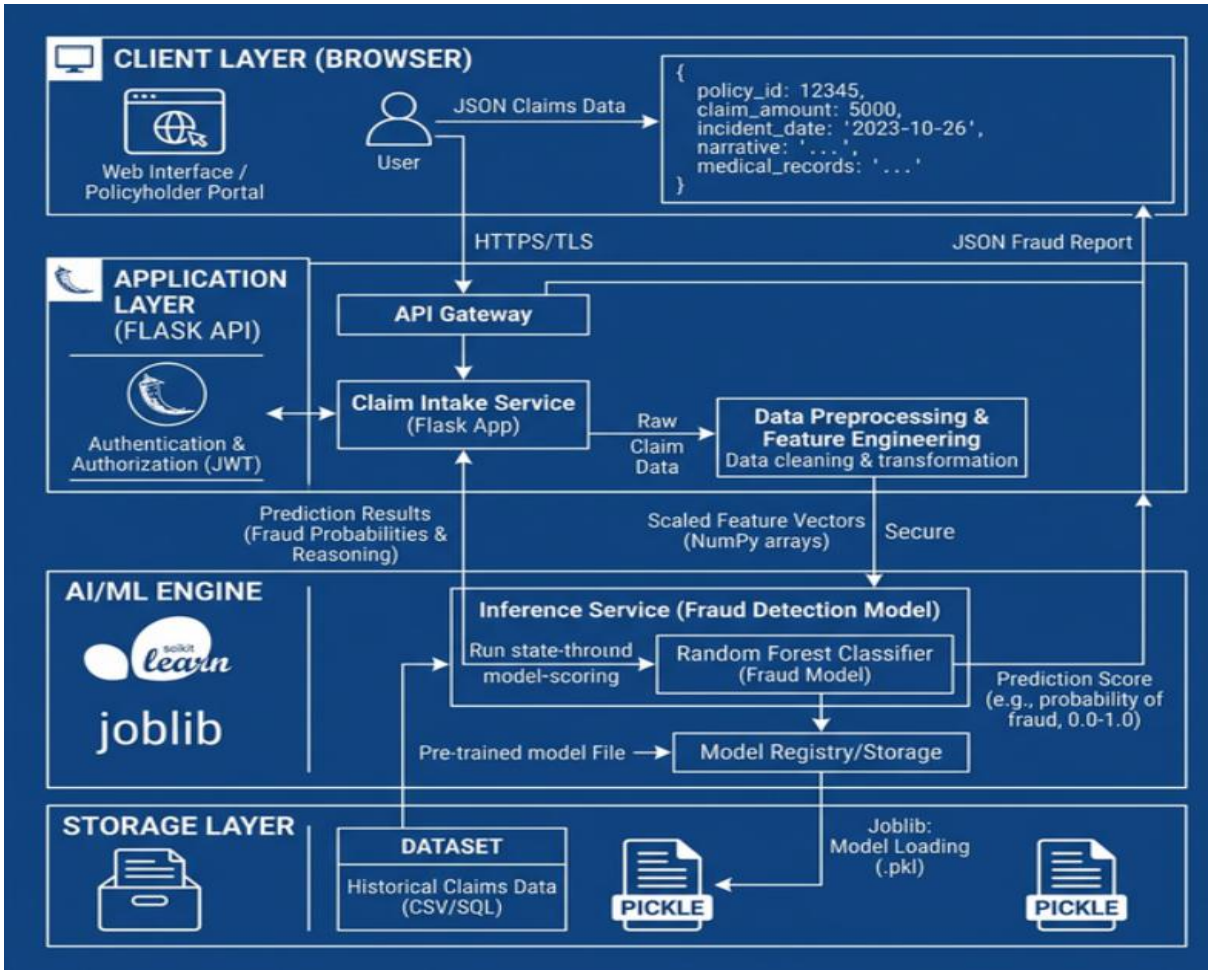

Table-1: Component & Technology Details:

S.No	Component	Description	Technology
1	**User Interface**	Web-based form for insurance agents to input claim data (age, incident severity, etc.).	HTML, CSS, JavaScript, Jinja2 Templates.
2	**Application Logic-1**	Core backend logic for handling HTTP requests and routing.	Python (Flask Framework).
3	**Application Logic-2**	Data preprocessing and feature transformation logic.	NumPy and Pandas.
4	**Machine Learning Model**	Binary classification model to predict Fraud (1) or Genuine (0).	Scikit-Learn (Random Forest Classifier).
5	**Database**	Local storage for training data and historical claim records.	CSV / SQLite.
6	**File Storage**	Storage for the serialized trained model file.	Local Filesystem (.pkl / pickle file).
7	**External API-1**	Visualization tools for the auditor's dashboard.	Matplotlib / Seaborn.
8	**Infrastructure**	Environment for running the local server and development.	VS Code, Python Virtual Environment.

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	**Open-Source Frameworks**	Utilizing widely supported open-source libraries for development.	Python, Flask, Scikit-Learn.
2	**Security Implementations**	Basic form validation and secure handling of user-submitted claim data.	Flask-WTF / CSRF Protection.
3	**Scalability**	Capable of handling increased claim volumes through modular code.	Modular Python Functions.
4	**Availability**	Localhost accessibility for immediate investigator use.	Flask Development Server.
5	**Performance**	Rapid inference time allowing real-time fraud flagging.	Optimized Scikit-Learn model.

References:

Flask Documentation, "Web development, one drop at a time," [Online]. Available: https://flask.palletsprojects.com/.

Scikit-Learn, "Random Forest Classifier Documentation," [Online]. Available: https://scikit-learn.org/stable/modules/ensemble.html.

Atlassian, "Agile Project Tracking - Burndown Charts," [Online]. Available: https://www.atlassian.com/agile/tutorials/burndown-charts.

Mural, "Empathy Map Canvas Template," [Online]. Available: https://www.mural.co/templates/empathy-map-canvas.